



Tecnológico de Monterrey

Evidencia 1 - Casos de Prueba

Daniel Gómez Guerrero A01572201

20 de septiembre del 2026

Estructura de Datos y Algoritmos Fundamentales (Gpo. 608)

David Alonso Cantú Delgado

Plan de Pruebas

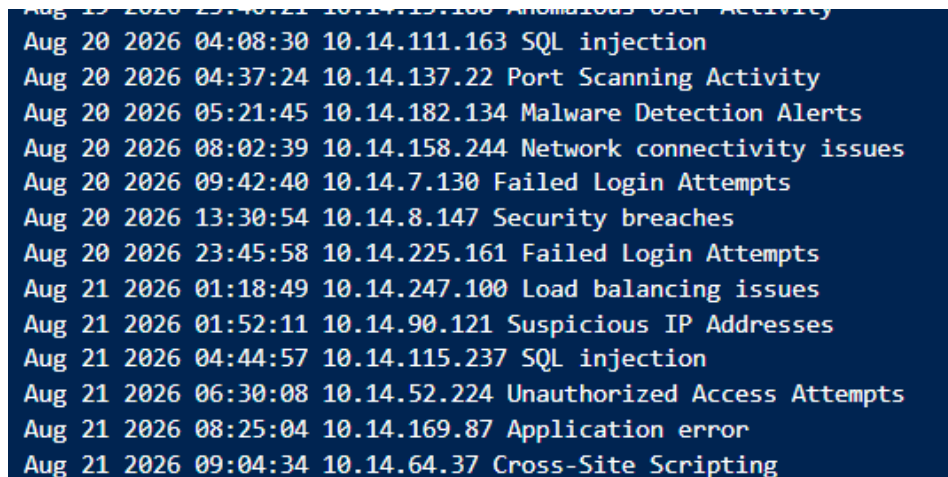
Durante el desarrollo de mi código, se tuvieron que realizar una gran cantidad de pruebas para asegurarse de que todo funcionara correctamente. Sin embargo, hubo un mayor enfoque en la búsqueda binaria, ya que el código de los algoritmos de ordenamiento ya había sido probado extensamente en la actividad anterior y en clase, por lo que solo se tuvo que adaptar el código ya realizado para que funcionara con logs.

Las pruebas que se validaron fueron las siguientes:

1. Verificar la lectura y apertura correcta de los archivos al crear las llaves (keys) utilizadas para ordenar los logs.
2. Verificar que el ordenamiento realizado fuera correcto.
3. Verificar si dejar vacío el rango de la búsqueda binaria generaba un error.
4. Introducir un rango correspondiente a una misma fecha y observar si se producía algún error.
5. Verificar si introducir la fecha o las horas en un formato incorrecto generaba algún error.
6. Verificar la presencia de timestamps duplicados.

Casos de Prueba Ejecutados

Prueba 1:



```
Aug 19 2026 23:40:21 10.14.137.100 Anomalous User Activity
Aug 20 2026 04:08:30 10.14.111.163 SQL injection
Aug 20 2026 04:37:24 10.14.137.22 Port Scanning Activity
Aug 20 2026 05:21:45 10.14.182.134 Malware Detection Alerts
Aug 20 2026 08:02:39 10.14.158.244 Network connectivity issues
Aug 20 2026 09:42:40 10.14.7.130 Failed Login Attempts
Aug 20 2026 13:30:54 10.14.8.147 Security breaches
Aug 20 2026 23:45:58 10.14.225.161 Failed Login Attempts
Aug 21 2026 01:18:49 10.14.247.100 Load balancing issues
Aug 21 2026 01:52:11 10.14.90.121 Suspicious IP Addresses
Aug 21 2026 04:44:57 10.14.115.237 SQL injection
Aug 21 2026 06:30:08 10.14.52.224 Unauthorized Access Attempts
Aug 21 2026 08:25:04 10.14.169.87 Application error
Aug 21 2026 09:04:34 10.14.64.37 Cross-Site Scripting
```

Al probar el código, los algoritmos de ordenamiento son capaces de ordenar correctamente el contenido del archivo elegido, lo que indica que los algoritmos de ordenamiento cumplen su función y que los archivos se abren de forma correcta.

Prueba 2:

```
Inserte la fecha de inicio (DD MM YYYY):  
00 00 0000  
Inserte la hora de inicio (hr:min:sec):  
0:0:0  
Inserte la fecha final (DD MM YYYY):  
00 00 0000  
Inserte la hora final (hr:min:sec):  
0:0:0  
Sep 08 2024 00:22:43 10.14.168.155 Unencrypted communication
```

El establecer como rango el valor mínimo posible, siendo 0 en todos los campos, parece resultar en el retorno del log que ocurrió más alejado en el tiempo. No fue realmente posible dejar el rango completamente vacío, ya que el código exige que se coloque un valor.

Prueba 3:

```
Inserte la fecha de inicio (DD MM YYYY):  
08 09 2024  
Inserte la hora de inicio (hr:min:sec):  
04:44:15  
Inserte la fecha final (DD MM YYYY):  
08 09 2024  
Inserte la hora final (hr:min:sec):  
04:44:15  
Sep 08 2024 04:44:15 10.14.135.12 Application error
```

El establecer exactamente la misma fecha como rango terminó resultando en que se retornara una sola fecha específica de los logs disponibles.

Prueba 4:

```
Inserte la fecha de inicio (DD MM YYYY):  
0092  
3445  
34445  
Inserte la hora de inicio (hr:min:sec):  
22344  
C:/msys64/ucrt64/include/c++/15.2.0/bits/st  
ring<char>; _Alloc = std::allocator<std::__  
>size()' failed.
```

El intentar establecer una fecha, especialmente una hora, fuera del formato indicado en el código resultó en un error. Esto indica que es absolutamente necesario seguir el formato indicado para que el código funcione.

Prueba 5:

```

Alerta de timestamp(s) duplicado(s). Se incluyen en el output todos los siguientes timestamps duplicados dentro del rango establecido en orden:
Oct 02 2024 23:04:24 10.14.121.223 Changes in File Integrity
Feb 13 2025 07:01:11 10.14.121.243 Data corruption
Mar 16 2025 16:58:11 10.14.187.166 Configuration errors
Apr 21 2025 21:45:23 10.14.68.110 Data corruption
Jun 05 2025 20:31:41 10.14.192.131 Network connectivity issues
Sep 26 2025 21:50:26 10.14.177.134 Cross-Site Scripting
Oct 08 2025 22:49:23 10.14.225.226 Failed Login Attempts
Nov 09 2025 02:42:01 10.14.215.213 Failed Login Attempts

```

Al correr el código, se le informa al usuario de manera exitosa sobre la presencia de timestamps duplicados en los logs y se procede a enlistarlos para que el usuario conozca su existencia.

Resultados de Tiempo

Swap Sort:

```

--- RESULTADOS ---
Algoritmo: Swap Sort
Archivo usado: log607-1.txt
Tamano del vector: 6818 elementos
Tiempo: 27394 milisegundos
Complejidad teorica: O(n^2)
Tu prediccion: 1000000
Razon: Es muy ineficiente
Tu prediccion difiere por 972606 ms.
-----

```

```

--- RESULTADOS ---
Algoritmo: Swap Sort
Archivo usado: log607-2.txt
Tamano del vector: 6818 elementos
Tiempo: 13543 milisegundos
Complejidad teorica: O(n^2)
Tu prediccion: 223
Razon: si
Tu prediccion difiere por 13320 ms.
-----

```

Bubble Sort:

```

--- RESULTADOS ---
Algoritmo: Bubble Sort
Archivo usado: log607-1.txt
Tamano del vector: 6818 elementos
Tiempo: 9086 milisegundos
Complejidad teorica: O(n^2)
Tu prediccion: 27000
Razon:
Tu prediccion difiere por 17914 ms.
-----

```

```

--- RESULTADOS ---
Algoritmo: Bubble Sort
Archivo usado: log607-2.txt
Tamano del vector: 6818 elementos
Tiempo: 19 milisegundos
Complejidad teorica: O(n^2)
Tu prediccion: 27000
Razon:
Tu prediccion difiere por 26981 ms.
-----

```

Selection Sort:

```

--- RESULTADOS ---
Algoritmo: Selection Sort
Archivo usado: log607-1.txt
Tamano del vector: 6818 elementos
Tiempo: 606 milisegundos
Complejidad teorica:  $O(n^2)$ 
Tu prediccion: 8000
Razon:
Tu prediccion difiere por 7394 ms.
-----

```

```

--- RESULTADOS ---
Algoritmo: Selection Sort
Archivo usado: log607-2.txt
Tamano del vector: 6818 elementos
Tiempo: 623 milisegundos
Complejidad teorica:  $O(n^2)$ 
Tu prediccion: 100
Razon:
Tu prediccion difiere por 523 ms.
-----

```

Insertion Sort:

```

--- RESULTADOS ---
Algoritmo: Insertion Sort
Archivo usado: log607-1.txt
Tamano del vector: 6818 elementos
Tiempo: 9019 milisegundos
Complejidad teorica:  $O(n^2)$ 
Tu prediccion: 100
Razon:
Tu prediccion difiere por 8919 ms.
-----

```

```

--- RESULTADOS ---
Algoritmo: Insertion Sort
Archivo usado: log607-2.txt
Tamano del vector: 6818 elementos
Tiempo: 588 milisegundos
Complejidad teorica:  $O(n^2)$ 
Tu prediccion: 3000
Razon:
Tu prediccion difiere por 2412 ms.
-----

```

Quick Sort:

```

--- RESULTADOS ---
Algoritmo: Quick Sort
Archivo usado: log607-1.txt
Tamano del vector: 6818 elementos
Tiempo: 44 milisegundos
Complejidad teorica:  $O(n \log n)$ 
Tu prediccion: 100
Razon:
Tu prediccion difiere por 56 ms.
-----

```

```

--- RESULTADOS ---
Algoritmo: Quick Sort
Archivo usado: log607-2.txt
Tamano del vector: 6818 elementos
Tiempo: 4000 milisegundos
Complejidad teorica:  $O(n \log n)$ 
Tu prediccion: 50
Razon:
Tu prediccion difiere por 3950 ms.
-----

```

Merge Sort:

```
--- RESULTADOS ---
Algoritmo: Merge Sort
Archivo usado: log607-1.txt
Tamano del vector: 6818 elementos
Tiempo: 78 milisegundos
Complejidad teorica:  $O(n \log n)$ 
Tu prediccion: 50
Razon:
Tu prediccion difiere por 28 ms.
-----
```

```
--- RESULTADOS ---
Algoritmo: Merge Sort
Archivo usado: log607-2.txt
Tamano del vector: 6818 elementos
Tiempo: 79 milisegundos
Complejidad teorica:  $O(n \log n)$ 
Tu prediccion: 50
Razon:
Tu prediccion difiere por 29 ms.
-----
```

Shell Sort:

```
--- RESULTADOS ---
Algoritmo: Shell Sort
Archivo usado: log607-1.txt
Tamano del vector: 6818 elementos
Tiempo: 63 milisegundos
Complejidad teorica:  $O(n \log n)$  a  $O(n^{1.5})$ 
Tu prediccion: 10
Razon:
Tu prediccion difiere por 53 ms.
-----
```

```
--- RESULTADOS ---
Algoritmo: Shell Sort
Archivo usado: log607-2.txt
Tamano del vector: 6818 elementos
Tiempo: 46 milisegundos
Complejidad teorica:  $O(n \log n)$  a  $O(n^{1.5})$ 
Tu prediccion: 20
Razon:
Tu prediccion difiere por 26 ms.
-----
```

Conclusión

Viendo las pruebas realizadas, se pudo notar que, sin lugar a dudas, entre los algoritmos de ordenamiento utilizados en el código, el más rápido en general fue Shell Sort, aunque Merge Sort no estaba tan alejado en cuanto a velocidad. En general, la mayoría de los algoritmos tuvieron resultados similares al ser ejecutados con los dos archivos de prueba proporcionados, siendo la mayoría más rápidos al ejecutarse con el archivo ya ordenado. Sin embargo, hubo algunos que, por alguna razón, fueron más lentos con el archivo ya ordenado. Las razones de esto podrían deberse a su forma específica de ordenar o tal vez a un fallo por parte de mi computadora o incluso del código que no haya detectado. Se requerirían pruebas más extensas para determinar la razón de esto.

Ahora, a través de las pruebas realizadas, se pudo observar que los algoritmos con mayor complejidad teórica eran significativamente más rápidos que aquellos con menor complejidad, siendo Swap Sort el algoritmo más lento de todos.

En general, la mayoría de las pruebas demostraron la funcionalidad de mi código. Sin embargo, no pude solucionar el error que se generaba al introducir la fecha y la hora en un formato incorrecto, por lo que este aspecto podría mejorarse.